

Hiding your Implementation Details is Not So Simple

Amir Kirsh

2024

About me

Lecturer

Academic College of Tel-Aviv-Yaffo
Tel-Aviv University

Member of the Israeli ISO C++ NB

Co-Organizer of the **CoreCpp**
conference and meetup group



Trainer and Advisor
(C++, but not only)



Hiding your Implementation Details

Hiding your Implementation Details

Why?

Encapsulation

Protect Object Integrity

Expose the necessary interfaces only.

Easier to Use

Users see only what they need to.

Improves Maintainability

Internal changes don't affect external code.

Easier to Debug

Data modifications happens internally, in specific places.

Decoupling

Reduces Dependencies

Components depend only what we actually expose.

Facilitates Changes

Changes in one part of the system don't necessarily affect others.

Enhances Reusability

Components become more generic, thus can be better reused.

Improves Testability

Easier to test components in isolation.

Hiding your Implementation Details *is Not So Simple*

Hiding your Implementation Details *is Not So Simple*

Why?

1. Because we're lazy and don't think it's that important*

“Lazy”, may include:

Time pressure: we have to cut the “fancy design” somewhere.

We don't think it's that important, may include:

We think we may change this later easily, if required
(Reality: in many cases you can't, not easily, if at all).

1. Because we're lazy and don't think it's that important

```
std::pair<First, Second> p;  
auto k = p.first;  
auto v = p.second;
```



Yes, there are other ways to extract first and second, but this one is still available and exposes private fields

I will try to convince you in this talk:

- Not to be lazy
- That it is important, even when it doesn't seem to be

2. Because we do not realize we expose private details

```
class Foo {  
    [...]  
public:  
    const std::map<string, int>& getProperties() const;  
};
```



What are the private details
exposed?

3. Because of technical constraints

```
class Foo {  
public:  
    // we prefer the ctor below to be private but it doesn't compile :(  
    Foo(int value) : value(value) {}  
    static unique_ptr<Foo> create(int value) {  
        return std::make_unique<Foo>(value);  
    }  
};  
  
auto foo = Foo::create(7);
```

Wait, don't jump, there are solutions
for this, we will discuss it later



⇒ Technical constraints can be addressed and resolved.

(Code: <https://compiler-explorer.com/z/Mvn9h3G88>)

4. Because there are tradeoffs

- We may want to avoid solutions relying on interfaces, to avoid virtual calls
- Exposing data to the user may be valuable for performance, e.g. exposing that we use a random access container, accessible with an index

Tradeoffs should be discussed:

- In some cases the achieved goal doesn't justify the design compromise.
- There may be a solution that meets both needs.

Our Goal

Investigate code examples:

- See how we expose implementation details
- Understand why it's not the best design, per case
- Propose a better design, hiding the implementation details

Let's start!

std::pair

std::pair

not hiding your privates is wrong

Data members should be private

`std::pair.first, std::pair.second` => is a *language accident...*

Why?

Because it doesn't properly allow different behaviors, e.g. a pair initialized with a single number, with the second being lazy evaluated to its square

std::pair

```
// assume function is constrained on PAIR having fields first and second
template<typename PAIR>
std::ostream& operator<<(std::ostream& out, const PAIR& pair) {
    return out << pair.first << ' ' << pair.second;
}

// ok
auto stdpair = std::make_pair("bye", 42);
std::cout << stdpair << std::endl;

// can we allow our pair to lazily evaluate its second?
auto squarepair = SquarePair(7);
std::cout << squarepair << std::endl;
```

(doable but not straightforward: <http://coliru.stacked-crooked.com/a/4c31320c394bcbb5>)

structs, in general

structs, in general

```
struct Order {  
    Price price;  
    Quantity quantity;  
};
```



What's the problem?

Supporting Generic Orders

Supporting Generic Orders

```
class Order {  
    Price price;  
    Quantity quantity;  
public:  
    // [ctor]  
    Price getPrice() const {  
        return price;  
    }  
    Quantity getQuantity() const {  
        return quantity;  
    }  
};
```

Supporting Generic Orders

```
class TimedOrder {  
    std::vector<chrono::time_point, Price> price_limits;  
    Quantity quantity;  
public:  
    // [ctor]  
    Price getPrice() const {  
        // fetch the proper price  
    }  
    Quantity getQuantity() const {  
        return quantity;  
    }  
};
```

We can treat this as a regular order, based on “Duck Typing”.



We rely here on “Static Polymorphism”.
But it may work also with “Dynamic Polymorphism”.



But if it's just a const member?

Just a const member...

```
class Person {  
    string name;  
public:  
    const long id;  
    Person(string name, long id)  
        : name(std::move(name)), id(id) {}  
};
```

What's the problem?



Just a const member...

```
class Person {  
    string name;  
public:  
    const Id id;  
    Person(string name, Id id)  
        : name(std::move(name)), id(id) {}  
};
```



Requirements tend to change:
A person may be identified by
SSN or Passport number.
The person shall hold an
internal Id object that would hold
either an SSN number or
passport details.

(What's the problem, still?)

Just a const member...

```
class Person {  
    string name;  
public:  
    const Id id;  
    Person(string name, Id id)  
        : name(std::move(name)), id(id) {}  
};
```

Requirements continue to change...



Just a const member - ... not any more

```
class Person {  
    string name;  
public:  
    const Id id;  
    Person(string name, Id id)  
        : name(std::move(name)), id(id) {}  
};
```



Requirements continue to change:
Passport number may change, when
issuing a new passport.
Person class shall allow that, keeping
the old passport numbers along with
the new one.

Person - as it should be to begin with

```
class Person {  
    string name;  
    Id id;  
public:  
    Person(string name, Id id)  
        : name(std::move(name)), id(id) {}  
    // expose relevant Id inquiries + relevant setters  
    // (e.g. newPassport(PassportDetails) etc.)  
};
```

Tony Van Eerd's Koan - The Master and his Dog

Tony Van Eerd's Koan - The Master and his Dog



Tony Van Eerd

@tvaneerd

...

Master, I am honoured by your visit.

I was out walking my dog. Your function, why does it take
BigCommonStruct instead of just x and y?

BCS is where we keep x and y, master.

Bark

He's hungry.

Shall I prepare him food?

No, just open the fridge, let him take what he wants.

[#CppKoan](#)

9:56 PM · Mar 23, 2018

Hyrum's Law

(“Software engineering at Google: lessons learned from programming over time”):

- Developers come to depend on all observable traits and behaviors of an interface, even if they are not defined in the contract.
- With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.

API (including internal API of all kinds!)

API

Studies show that applications tend to use a small part of an API.

API

Studies show that applications tend to use a small part of an API.

Unused API is much more bug prone!

API

Studies show that applications tend to use a small part of an API.

Unused API is much more bug prone!

When there are too many options (e.g. too many overloads), a wrong option would be selected at some point.

Be Lean and Mean

1. Keep your API short and concise.
(add as needed, and only for there is a real need).
2. Limit your API to what should actually be used
(arguments, return value etc.) - do not expose too much of your design!
3. Different usages may get different *view* (or copy) of the same data, exposing only what is relevant.

It's context specific!

It's context specific!

Something that is reasonably exposed in a certain context may need to be considered “private” in another context!

⇒ The interface that we provide shall be as narrow as possible.

- Can we expose different behavior for different contexts?
- Can we identify the different contexts?

The Master and his Dog

The Master and his Dog



Tony Van Eerd

@tvaneerd

...

Master, I am honoured by your visit.

I was out walking my dog. Your function, why does it take
BigCommonStruct instead of just x and y?

BCS is where we keep x and y, master.

Bark

He's hungry.

Shall I prepare him food?

No, just open the fridge, let him take what he wants.

[#CppKoan](#)

9:56 PM · Mar 23, 2018

The Master and his Dog - Act II



Amir Kirsh
@AmirKirsh

Master, tho I trust your dog completely, I still got a concern.
What's your concern? the fridge engine is in the private.

It's context specific!

Something may be required to be public for one usage.

Yet, have no reason to be public for other usages.

It's context specific!

Something may be required to be public for one usage.

Yet, have no reason to be public for other usages.

Because:

- If they don't need it, don't give it.
- If they get it, they'll use it.
- If they use it, they'll abuse it.
- Trust no one!

It's context specific!

Something may be required to be public for one usage.

Yet, have no reason to be public for other usages.

Because:

- If they don't need it, don't give it.
- If they get it, they'll use it.
- If they use it, they'll abuse it.
- Trust no one!



Having “things” in the private is not enough!

Exercise - “Const Map, Mutable Vals”

Initialize a map, then pass it in a way such that values can be modified but the map itself cannot be modified.

Exercise - “Const Map, Mutable Vals” - Attempt 1

```
// note: function may modify values but should not alert the map itself!
void foo(Dictionary auto& d) {
    d[1] = 3;
}

int main() {
    map<int, int> myMap = {{1, 0}, {2, 0}};
    foo(myMap);
    for (const auto& [k, v] : myMap) {
        std::cout << k << ": " << v << std::endl;
    }
}
```

Exercise - “Const Map, Mutable Vals” - Attempt 2

```
// note: function may modify values but should not alert the map itself!  
void foo(Dictionary auto& d) {  
    auto itr = d.find(1);  
    if(itr != d.end()) itr->second = 3;  
}  
  
int main() {  
    map<int, int> myMap = {{1, 0}, {2, 0}};  
    foo(myMap);  
    for (const auto& [k, v] : myMap) {  
        std::cout << k << ": " << v << std::endl;  
    }  
}
```

Exercise - “Const Map, Mutable Vals” - Attempt 3

// note: function may modify values but cannot alert the map itself!

```
template<Dictionary Dict>
void foo(ConstDictMutableValues<Dict> d) {
    d.assign(1, 3);
}

int main() {
    map<int, int> myMap = {{1, 0}, {2, 0}};
    foo(ConstDictMutableValues(myMap));
    for (const auto& [k, v] : myMap) {
        std::cout << k << ": " << v << std::endl;
    }
}
```

ConstDictMutableValues - A Wrapper

```
template<Dictionary Dict>
class ConstDictMutableValues {
    Dict& d_;
public:
    explicit ConstDictMutableValues(Dict& d) : d_(d) {}
    bool assign(const Key& key, Value value) {
        auto itr = d_.find(key);
        if(itr == d_.end()) return false;
        itr->second = std::move(value);
        return true;
    }
};
```



A wrapper class providing only
the required functionality

Code: <https://coliru.stacked-crooked.com/a/163f4dc6e2974f77>

Hiding Smart Pointers and Hierarchies

Math Expression Hierarchy

// we want the following code:

```
Expression* e = new Sum (  
    new Exp(new Number(3), new Number(2)),  
    new Number(-1)  
);  
cout << *e << " = " << e->eval() << endl;  
delete e;
```

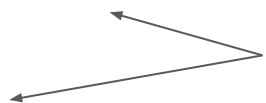
// to print something like:

// ((3 ^ 2) + (-1)) = 8

But..., do you like it?

// what is bothering you with the code below?

```
Expression* e = new Sum (  
    new Exp(new Number(3), new Number(2)),  
    new Number(-1)  
);  
cout << *e << " = " << e->eval() << endl;  
delete e;
```

 Non-symmetric calls to new and delete

Why do we use new and delete to begin with?
Shouldn't we use smart pointers? 🤔

Let's move to smart pointers!

// what is bothering you with the code below?

```
auto e = make_unique<Sum>(  
    make_unique<Exp>(  
        make_unique<Number>(3),  
        make_unique<Number>(2)  
    ),  
    make_unique<Number>(-1)  
);  
cout << *e << " = " << e->eval() << endl;
```

Let's try to hide the `unique_ptr`

We aim for something like this

```
auto e = Sum(Exp(Number(3), Number(2)), Number(-1));  
cout << e << " = " << e.eval() << endl;
```

Why is it better?

```
auto e = Sum(Exp(Number(3), Number(2)), Number(-1));  
cout << e << " = " << e.eval() << endl;
```

// what makes the code above better? compared to:

```
auto e = make_unique<Sum>(
    make_unique<Exp>(make_unique<Number>(3), make_unique<Number>(2)),
    make_unique<Number>(-1)
);  
cout << *e << " = " << e->eval() << endl;
```

Hiding your implementation details!

The user doesn't have to know we use `unique_ptr`

We may want to change it later

It's a "private implementation detail"

And it's noisy

Code: <http://coliru.stacked-crooked.com/a/35f49a5a014224f8>

Talk: [Six Ways for Implementing Math Expressions Calculator in C++ - Amir Kirsh - CppCon 2023](#)

Hiding Inheritance Hierarchies Details

Using:

- **State Pattern**

(Employee is an Employee, regardless of his employment type - a state)

- **Strategy Pattern**

(A PathFinder is a PathFinder, regardless if using BFS or DFS - a strategy)

- **Factory Method**

(The user shall not be bothered with the exact object type instantiated)

Conveying less information on return values

Conveying less information on return values

```
// what do you say about this one?  
vector<int> foo() {  
    return vector {1, 2, 4};  
}
```

Conveying less information on return values

```
// is this one better?  
auto foo() {  
    return vector {1, 2, 4};  
}
```

Conveying less information on return values

```
// and what about this one?  
random_access_range_of<int> auto foo2() {  
    return vector {1, 2, 4};  
}
```

Code: <https://coliru.stacked-crooked.com/a/e07ac6d370cbbfed>

Conveying less information on return values

Using as a return value type:

- **A concept**

- => Since C++20.

- => Only for functions implemented in the header (auto must be deduced).

- => Compiler doesn't check if *usage* conforms to the concept...



Back to the hazards pointed by
Hyrum's Law.

Yes, that's a real problem. See slide 60
in Dave Abrahams keynote talk.

Would static code analysis tools point
at such issues?

Conveying less information on return values

Using as a return value type:

- **A concept**

- => Since C++20.

- => Only for functions implemented in the header (auto must be deduced).

- => Compiler doesn't check if *usage* conforms to the concept...

- **An interface**

- => Using dynamic virtual calls..



Back to the hazards pointed by
Hyrum's Law.

Yes, that's a real problem. See slide 60
in Dave Abrahams keynote talk.

Would static code analysis tools point
at such issues?

Conveying less information on return values

Using as a return value type:

- **A concept**

- => Since C++20.

- => Only for functions implemented in the header (auto must be deduced).

- => Compiler doesn't check if *usage* conforms to the concept...

- **An interface**

- => Using dynamic virtual calls.

- **A wrapper class**

- => A bit more work.



Back to the hazards pointed by Hyrum's Law.

Yes, that's a real problem. See slide 60 in Dave Abrahams keynote talk. Would static code analysis tools point at such issues?

Hiding implementation details of return value

A note:

Beware of “leaky abstractions”

(Abstractions that expose to the user the internal behavior of the abstraction itself)

Hiding implementation details of return value

A note:

Beware of “leaky abstractions”

(Abstractions that expose to the user the internal behavior of the abstraction itself)

An example?

Hiding implementation details of return value

A note:

Beware of “leaky abstractions”

(Abstractions that expose to the user the internal behavior of the abstraction itself)

An example?

```
std::vector<bool>
```

(^ the proxy returned from)

Hiding Helper Functions

Hiding Helper Functions

```
// what do say about this one?  
class Thingy {  
public:  
    void foo();  
    // note: do not call bar before calling foo!  
    void bar();  
};
```

Hiding Helper Functions - An Attempt

```
// would this be a valid solution?  
class Thingy {  
    void foo();  
    // note: do not call bar before calling foo!  
    void bar();  
public:  
    void foobar() { foo(); bar(); }  
};
```

Hiding Helper Functions - More Generic

```
// foo and bar are related, but in a more generic way
class Thingy {
    void foo();
    // note: do not call bar before calling foo!
    void bar();
public:
    template<typename Func> void foobar(Func&& func) {
        foo();
        if(func()) bar();
    }
};
```

Code: <https://coliru.stacked-crooked.com/a/a0f86e35b362b333>

Hiding Helper Functions - Yet another approach

// Credit: Jonathan Storey - come to his Friday talk: Coroutines and gRPC

```
class Thingy {  
    void inner_foo() { std::cout << "foo" << std::endl; }  
    void bar() { std::cout << "bar" << std::endl; }  
public:  
    auto foo() {  
        inner_foo();  
        struct BarCallable {  
            Thingy* t;  
            void bar() { t->bar(); }  
        };  
        return BarCallable(this);  
    }  
};
```

```
int main() {  
    Thingy t;  
    t.foo().bar();  
    t.foo();  
}
```

Code: <https://coliru.stacked-crooked.com/a/c8cefb4eb38c9780>

Hiding Helper Functions

API should be hard to misuse

Using “protected”

Using “protected”

// what do say about this?

```
class Parent {  
protected:  
    Wallet wallet;  
};
```

```
class Child: public Parent {  
public:  
    void endless_celebration(long double) { wallet.draw(...); }  
};
```

Using “protected”

// A much better approach

```
class Parent {  
    Wallet wallet;  
protected:  
    short need_money(long double, const std::string& reason);  
};  
  
class Child: public Parent {  
public:  
    void endless_celebration(long double) { need_money(...); }  
};
```

Using “protected”

Strongly prefer *data members* to be **private**.

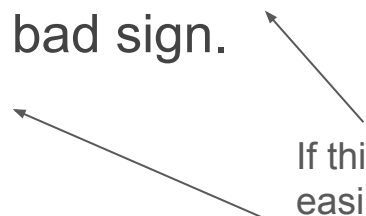
Prefer **protected *member functions*** over public, if possible.

Testing private behavior

Testing private behavior (1)

- Usually testing private behavior (e.g. the class is in state A) should be avoided.
- If there is a real need to test that, do not turn data members to be public for that purpose! Use proper member functions.
- Adding a public member function for validating class state can be useful also for logging and traceability.

Testing private behavior (2)

- A need to change the state of a class, by the test itself, not through regular flow, is usually a bad sign: we do not test actual flows (need to test error scenarios? Use mocking, create the error case via your flow).
 - Changing the code to fit your test is not a bad sign. It may improve our code!
 - Test the manipulates private data may be unstable and shaky. And usually would not test real life scenarios.
- 
- If this cannot be done easily, it may mean that your code needs redesign for better decoupling.

Testing private behavior (3)

If you still need to test private members, there are techniques for doing it, without turning them to be public.

This should not be your first choice and we will not cover it here.

Use the proper context for your code

Use the proper context for your code

- Use namespaces properly to give context to your code, including hidden “detail” and unnamed (anonymous) namespaces.
- Use nested type: put types (classes, enums etc.) inside classes when relevant. Hide them in the private section if possible.
- Remember that a type declared in the private part can still be used externally, if an object of that type is provided (useful for example for inner proxies).

Last Item...

Last Item...

I owe you something.

Do you remember?

Private Token Idiom

Private Token Idiom

AKA:

- Access Token Pattern
- Passkey Pattern
- Authorization Token Pattern
- Key Token Idiom
- ...

On the naming, see:

<https://stackoverflow.com/questions/3324248/how-to-name-this-key-oriented-access-protection-pattern>

Private Token Idiom

Remember this:

```
class Foo {  
public:  
    // we prefer the ctor below to be private but it doesn't compile :(  
    Foo(int value) : value(value) {}  
    static unique_ptr<Foo> create(int value) {  
        return std::make_unique<Foo>(value);  
    }  
};  
  
auto foo = Foo::create(7);
```

Can we make the ctor
private and still use
make_unique?

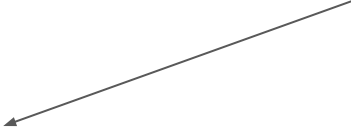


Code: <https://compiler-explorer.com/z/Mvn9h3G88>

Private Token Idiom

```
class Foo {  
    int value;  
    class PrivateToken {};  
public:  
    Foo(int value, PrivateToken) : value(value) {}  
    static unique_ptr<Foo> create(int value) {  
        return std::make_unique<Foo>(value, PrivateToken{});  
    }  
    int getValue() const { return value; }  
};  
  
auto foo = Foo::create(7);
```

The ctor is still in the public, but usable only with a private token.



Code: <https://compiler-explorer.com/z/M8Yof53e5>

Summary

Summary (1)

Hiding your implementation details is more than just a design recommendation for the protocol.

It will make your code **more robust** and **maintainable**.

Summary (2)

Hiding rules go well with known design principles:

Encapsulation:

Protecting Object Integrity.

Exposing necessary interfaces only.

Internal changes don't affect external code.

Easier to Debug.

Decoupling:

Reduces Dependencies: components depend only what we actually expose.

Changes in one part of the system don't necessarily affect others.

Components become more generic.

Improves Testability

Summary (3)

Having private fields is NOT enough!

(Mandatory but not sufficient).

Implementation details is context specific.

Something that should be public for a certain context should not be exposed in another.

It's context specific!

Something may be required to be public for one usage.

Yet, have no reason to be public for other usages.

Because:

- If they don't need it, don't give it.
- If they get it, they'll use it.
- If they use it, they'll abuse it.
- Trust no one!



Having “things” in the private is not enough!

Summary (4)

You may have reasons why to avoid hiding implementation details. 95% of the reasons are wrong.

Look for things that shouldn't be exposed.

This will improve your code and save you time and money by preventing future defects and refactoring.

Any questions before we conclude?



Bye

```
class Greetings {  
    std::string greetings[] =  
        {"Thank you!"s, "תודה לכולם"s};  
public:  
    void greet() const;  
};
```